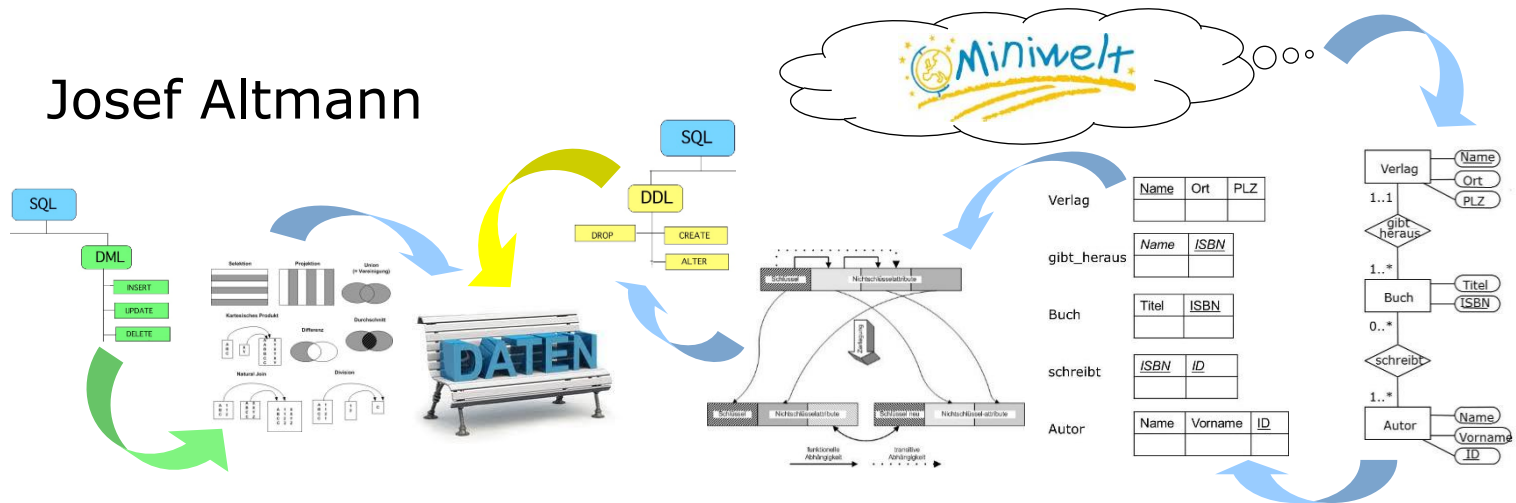


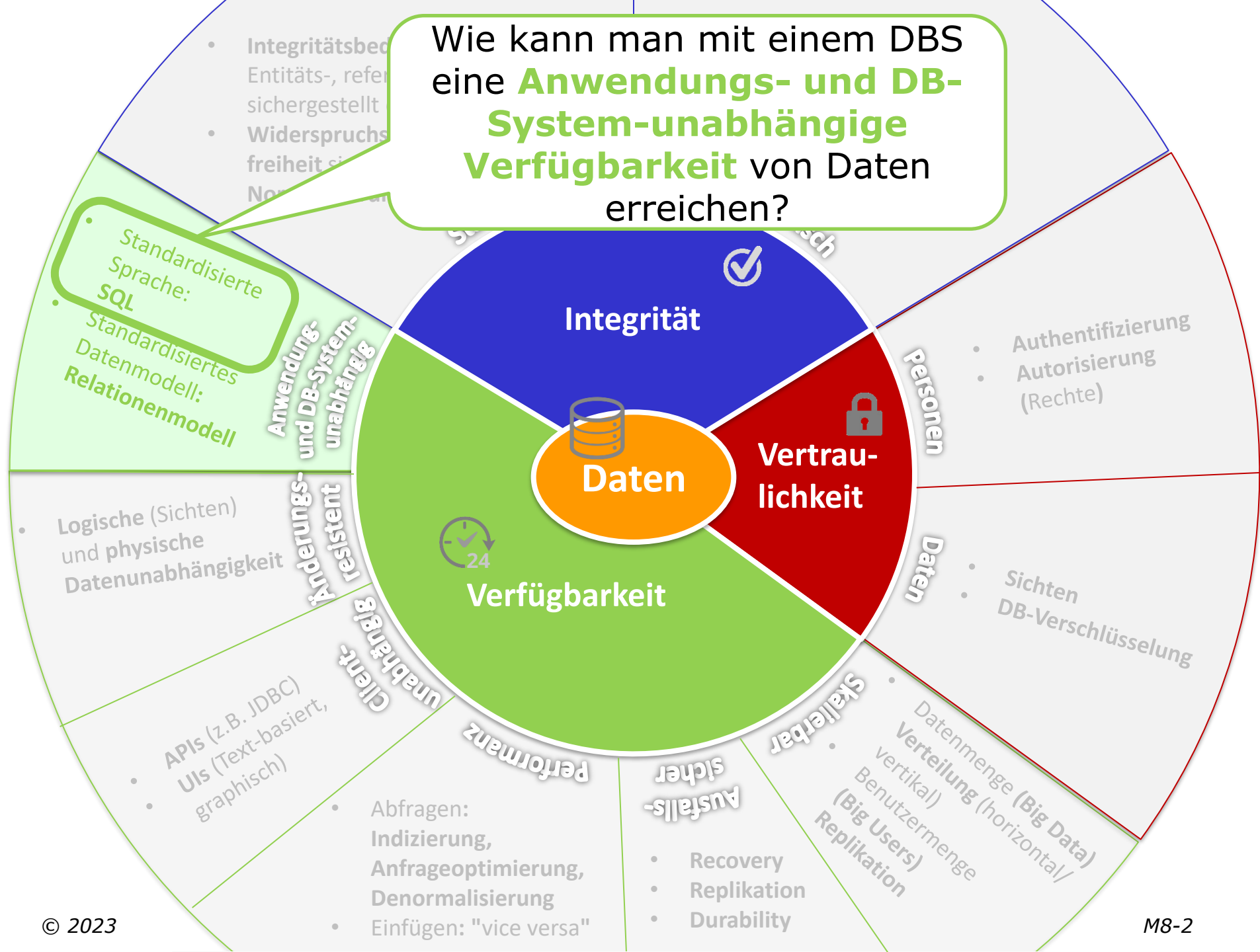
Modul 8

SQL-Datenmanipulationssprache

Josef Altmann



Wie kann man mit einem DBS eine **Anwendungs- und DB-System-unabhängige Verfügbarkeit** von Daten erreichen?



Überblick

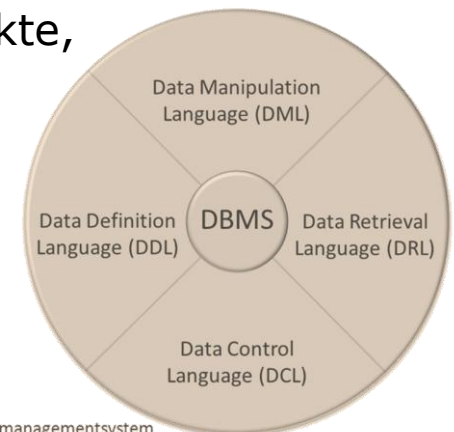
- Einführung Datenmanipulationssprache SQL
- Manipulationsoperationen
 - **UPDATE**-Anweisung
 - **DELETE**-Anweisung
 - **INSERT**-Anweisung
 - **MERGE**-Anweisung

Structured Query Language

- SQL stellt eine standardisierte
 - **Datendefinitionssprache** (DDL)
 - Erstellen, Ändern und Löschen von Datenbankobjekten (Tabellen, etc.)
 - **Datenmanipulationssprache** (DML)
 - Einfügen, Ändern und Löschen von Daten
 - **Anfrage (Query)-Sprache** (DRL)
 - Abfragen von Daten (dies ist der komplexeste Teil von SQL)
 - **Kontrollsprache** (DCL)
 - Verwalten von Zugriffsrechte auf Datenbankobjekte, Transaktionskontrolle, etc.

bereit.

- SQL ist
 - **deklarativ**
 - **mengenorientiert**



DBMS =
Datenbankmanagementsystem

Datenmanipulation

Structured Query Language

- Bisher: Nur Anfragen an das DBS
- Änderungsoperationen **modifizieren** den Inhalt **eines** oder **mehrerer Tupeln** einer Relation:
 - **INSERT**: Einfügen eines oder mehrerer Tupeln
 - **UPDATE**: Ändern von einem oder mehreren Tupeln
 - **DELETE**: Löschen eines oder mehrerer Tupeln
 - **MERGE**: Kombination von **INSERT**, **UPDATE** und **DELETE** (seit SQL:2003)
- **Integritätsbedingungen** werden bei Änderungsoperationen automatisch vom System überprüft
- Verletzung von Integritätsbedingungen führen zum **Abbruch** der Operation mit Fehlermeldung

INSERT-Anweisung 1/5

Einfügen konstanter Tupeln

Notation

```
INSERT  
INTO tabelle  
[ (attribut1, ..., attributn) ]  
VALUES (konstante1, ..., konstanten);
```

- **Optionale Attributliste** ermöglicht das **Einfügen** von **unvollständigen Tupeln**
 - nicht aufgefüllte Attribute werden auf **NULL** gesetzt

INSERT-Anweisung 2/5

Einfügen konstanter Tupeln

- Realisierung von Ein-Tupel-Operation mittels Primärschlüssel und Attributliste:

Beispiel

```
INSERT  
INTO Professor (PersNr, Name, Rang)  
VALUES (2140, 'Altmann', 'C4');
```

- Nicht alle Attribute angegeben
 - Wert des fehlenden Attributes **Raum** wird **NULL** (bzw. Default-Wert)
- Realisierung von Ein-Tupel-Operation mittels Primärschlüssel und ohne Attributliste

Beispiel

```
INSERT  
INTO Professor  
VALUES (2140, 'Altmann', 'C4', 999);
```

INSERT-Anweisung 3/5

Einfügen berechneter Tupeln

Notation

```
INSERT  
INTO tabelle  
[ (attribut1, ..., attributn) ]  
SQL-Anfrage;
```

- Alle Tupel des Ergebnisses der SQL-Anfrage werden in die Tabelle eingefügt

Beispiel

```
INSERT  
INTO hoeren (MatrNr, VorlNr)  
  ( SELECT MatrNr, 1001  
    FROM Student  
    WHERE Semester < = 2 );
```


INSERT-Anweisung 4/5

Einfügen mehrerer Tupeln

Notation

```
INSERT  
INTO tabelle  
[ (attribut1, ..., attributn) ]  
VALUES (konstante1, ..., konstanten)  
      [, (konstante1, ..., konstanten), ... ];
```

- Alle Tupel der Werteliste werden in die Tabelle eingefügt

Beispiel

```
INSERT  
INTO Professor (PersNr, Name, Rang)  
VALUES (2140, 'Altmann', 'C4'),  
       (2141, 'Augstein', 'C3'),  
       (2142, 'Jadin', 'C4');
```

INSERT-Anweisung 5/5

Einfügen mehrerer Tupeln - ORACLE

- Oracle-Anweisung **INSERT ALL** wird verwendet, um mehrere Tupel mit einer einzigen **INSERT**-Anweisung hinzuzufügen.

Beispiel (Oracle)

```
INSERT ALL  
INTO Professor (PersNr, Name, Rang) VALUES (2140, 'Altmann', 'C4')  
INTO Professor (PersNr, Name, Rang) VALUES (2141, 'Augstein', 'C3')  
INTO Professor (PersNr, Name, Rang) VALUES (2142, 'Jadin', 'C4')  
SELECT * FROM dual;
```

- Die **INSERT ALL**-Anweisung kann auch dazu verwendet werden, mehrere Zeilen in mehr als eine Tabelle in einem Befehl einzufügen.
- Bedingungsabhängige **INSERT ALL**-Anweisung siehe Anhang!

UPDATE-Anweisung 1/3

Verändern von Tupeln

Notation

```
UPDATE tabelle  
SET attribut1 = ausdruck1  
[, ...,  
attributn = ausdruckn]*  
[ WHERE bedingung ];
```

- Erfolgt keine Angabe hinter der **WHERE**-Klausel, erfolgt eine Änderung der gesamten Relation

UPDATE-Anweisung 2/3

Verändern von Tupeln

Beispiel

Erhöhen Sie die Semesteranzahl der Studierenden bei Semesterbeginn.

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	18
25403	Jonas	12
26120	Fichte	10
26830	Aristoxenos	8
27550	Schopenhauer	6
28106	Carnap	3
29120	Theophrastos	2
29555	Feuerbach	2

UPDATE Student

SET Semester = Semester + 1;

Mehr-Tupel-Operation

Student

<u>MatrNr</u>	Name	Semester
24002	Xenokrates	19
25403	Jonas	13
26120	Fichte	11
26830	Aristoxenos	9
27550	Schopenhauer	7
28106	Carnap	4
29120	Theophrastos	3
29555	Feuerbach	3

UPDATE-Anweisung 3/3

Verändern von Tupeln

- Realisierung von Ein-Tupel-Operation mittels Primärschlüssel

Beispiel

```
UPDATE Student  
SET Semester = Semester + 1  
WHERE MatrNr = 24002;
```

DELETE-Anweisung 1/4

Löschen von Tupeln

Notation

```
DELETE  
FROM tabelle  
[ WHERE bedingung ];
```

Löschen eines Tupels in der `student`-Tabelle

```
DELETE  
FROM Student  
WHERE MatrNr = 24002;
```

■ DELETE versus DROP:

- **DELETE FROM Student** löscht alle Tupel in der `Studenten`-Tabelle, das Schema bleibt jedoch im Data Dictionary erhalten
- **DROP TABLE Student** löscht alle Tupel, Schemadefinition, Indices usw. der Tabelle `Student`

DELETE-Anweisung 2/4

Löschen von Tupeln

Löschen mehrerer Tupeln in der **student**-Tabelle

```
DELETE  
FROM Student  
WHERE Semester > 13;
```

Löschen aller Tupeln in der **student**-Tabelle

```
DELETE  
FROM Student;
```

DELETE-Anweisung 3/4

Löschen von Tupeln

- Löschoperationen können zur **Verletzung** von **Integritätsbedingungen** führen!
- Beispiel
 - Verletzung der Fremdschlüsseleigenschaft, falls es noch Vorlesungen von diesem Professor gibt.

Beispiel

```
DELETE  
FROM Professor  
WHERE PersNr = 2125;
```


DELETE-Anweisung 4/4

Löschen von Tupeln

- Zeilen basierend auf einer anderen Tabelle löschen:

Löschen aller Prüfungen von Professor **Russel**

```
DELETE
FROM pruefen
WHERE PersNr =
      ( SELECT PersNr
        FROM Professor
        WHERE Name = 'Russel' );
```

Löschen aller Studierenden, die keine Vorlesung hören.

```
DELETE
FROM Student
WHERE MatrNr NOT IN
      ( SELECT MatrNr
        FROM hoeren );
```

MERGE-Anweisung 1/3

- Seit SQL:2003
- Kombination von mehreren **UPDATE**- oder **DELETE**- sowie **INSERT**-Anweisungen in einer Anweisung
- **Bedingungsabhängiges Aktualisieren oder Löschen bzw. Einfügen von Spaltenwerten bzw. Tupel:**
 - **UPDATE**- oder **DELETE**-Anweisung, wenn Zeile schon vorhanden ist
 - **INSERT**-Anweisung, wenn Zeile noch nicht vorhanden ist
- Vorteile:
 - Vermeidet separate Aktualisierungen (Synchronisation von Tabellen)
 - Steigert Performance und Benutzerfreundlichkeit
 - Nützlich für Data Warehousing-Anwendungen (Ladevorgang kann damit beschleunigt werden)

MERGE-Anweisung 2/3

Zieltabelle, deren Daten aktualisiert, gelöscht oder ergänzt werden

Notation

```

MERGE    INTO zieltabelle
           USING referenztable | SQL-Anfrage
           ON (verbundbedingung)
WHEN MATCHED THEN
    { UPDATE SET attribut1 = wert1 [, attribut2 = wert2 , ...]
      WHERE (bedingung)
      [DELETE WHERE (bedingung)]
WHEN NOT MATCHED THEN
    { INSERT (attribut1 [, attribut2 , ...])
      VALUES (wert1 [, wert2 , ...])
      WHERE (bedingung);
  
```

Aktualisiere/lösche bestehenden Tupel

Quelle kann Tabelle, View oder Unteranfrage sein

Füge neuen Tupel ein

- Mindestens eine der beiden **MATCHED**-Klauseln muss angegeben werden.
- Je nachdem, ob die Verbundbedingung erfüllt ist oder nicht, wird die jeweilige Klausel ausgeführt.

MERGE-Anweisung 3/3

Beispiel: Synchronisation der Prüfungsstatistik

```
MERGE INTO Pruefungsstatistik ziel
USING (
    SELECT PersNr, COUNT(*) AS PrfAnzahl
    FROM pruefen
    GROUP BY PersNr) quelle
ON (ziel.PersNr = quelle.PersNr)
```

Pruefungsstatistik

PersNr	PrfAnzahl
2137	2
...	...

```
WHEN MATCHED THEN
    UPDATE SET ziel.PrfAnzahl = quelle.PrfAnzahl
DELETE WHERE ziel.PersNr=2126
```

```
WHEN NOT MATCHED THEN
    INSERT (PersNr, PrfAnzahl)
    VALUES (quelle.PersNr, quelle.PrfAnzahl)
WHERE NOT (quelle.PersNr=2126);
```

MERGE-Anweisung

Systemunterstützung

SQL-2021 WHEN ... THEN-REGELN	Oracle	DB2	SQL Server	PostgreSQL
WHEN NOT MATCHED THEN INSERT	✓	✓	✓	✓
WHEN MACHED THEN UPDATE	✓ ^a	✓	✓	✓
WHEN MACHED THEN DELETE	– ^b	✓	✓	✓

^a Kein Update auf Spalten, die in der **ON**-Klausel genannt werden

^b Nicht zu verwechseln mit **THEN UPDATE ... DELETE [WHERE]**

Umfangreiche Erweiterungen (Syntax-Varianten) in Datenbanksystemen vorhanden!
[siehe: <https://modern-sql.com/de/gehtdas/merge>]

Überblick

- Einführung Datenmanipulationssprache SQL
- Manipulationsoperationen
 - **UPDATE**-Anweisung
 - **DELETE**-Anweisung
 - **INSERT**-Anweisung
 - **MERGE**-Anweisung



Anhang

ORACLE

INSERT ALL

Bedingungsabhängiges Einfügen

INSERT ALL-Anweisung 1/2

Bedingungsabhängiges Einfügen

Notation

INSERT ALL | FIRST
erfordert SQL-Anfrage

```
INSERT [ ALL | FIRST ]  
  WHEN bedingung_1 THEN  
    INTO tabelle_1(attribut_liste_1) VALUES (werte_liste_1)  
  WHEN bedingung_2 THEN  
    INTO tabelle_2(attribut_liste_2) VALUES (werte_liste_2)  
  ELSE  
    INTO tabelle_3(attribut_liste_3) VALUES (werte_liste_3)  
[SQL-Anfrage;
```

- **ALL**: alle **WHEN**-Klauseln werden ausgewertet
- **FIRST**: **WHEN**-Klauseln werden top-down ausgewertet, bei **TRUE** werden nachfolgende **WHEN**-Klauseln ignoriert
- SQL-Anfrage erforderlich: **SELECT** * **FROM** dual;

INSERT ALL-Anweisung 2/2

Bedingungsabhängiges Einfügen

Beispiel:

```

INSERT ALL
  WHEN amount < 10000 THEN
    INTO small_orders(order_id, customer_id, amount)
    VALUES(order_id, customer_id, amount)
  WHEN amount >= 10000 AND amount <= 30000 THEN
    INTO medium_orders(order_id, customer_id, amount)
    VALUES(order_id, customer_id, amount)
  WHEN amount > 30000 THEN
    INTO big_orders(order_id, customer_id, amount)
    VALUES(order_id, customer_id, amount)
SELECT order_id, customer_id, (quantity * unit_price) amount
FROM orders
INNER JOIN order_items USING(order_id);
  
```

orders

order_id	customer_id	amount
...	...	...

order_items

order_id	item_id	product_id	quantity	unit_price
...	...	...	...	...

small_orders

order_id	customer_id	amount
...	...	...

medium_orders

order_id	customer_id	amount
...	...	...

big_orders

order_id	customer_id	amount
...	...	...